Homework #2

Vanja Stojanović

April 2026

1 Where is SN100?

We study the given bottlenose dolphine network. Given that the disappearance of the dolphine SN100, yields distinct communities, it is natural to explore its centrality measures. In particular the degree, betweenness, closeness, and eigenvector centralities. With these we can detected how important SN100 was structurally, i.e. seeing if it formed bridges between communities. Results of the measures can be seen in Table 1 below.

Table 1: Top 5 dolphins by each centrality measure. SN100 is **bolded** where it appears.

	Betweenness			*Closeness*	
Rank	Dolphin	Value	Rank	Dolphin	Value
1	**SN100**	0.248	1	**SN100**	0.418
2	Beescratch	0.213	2	SN9	0.404
3	SN9	0.143	3	SN4	0.399
4	SN4	0.139	4	Kringel	0.391
5	DN63	0.118	5	Grin	0.377
	Degree			*Eigenvector*	
Rank	Dolphin	Value	Rank	Dolphin	Value
1	Grin	0.197	1	Grin	0.316
2	SN4	0.180	2	SN4	0.301
3	Topless	0.180	3	Topless	0.285
4	Scabs	0.164	4	Scabs	0.281
5	Trigger	0.164	5	TR99	0.218

SN100 ranks 1st in both betweenness and closeness centrality despite only ranking 17th in degree ($k = 7$). This confirms its role as broker, it bridges otherwise

distant sub-communities rather than being a highly connected hub. Its low eigenvector centrality (rank 19) further indicates that its importance comes from its structural position, not from being connected to other hubs. We can further confim this by looking at the neighbourhood of SN100: Beescratch, Kringel, MN60, SN4, SN89, SN9, Zap. These neighbours also appear high in the measures given in the Table 1 above, in particular SN4 is rank 2 in degree, and eigenvector centrality.

The following is the Python implementation used to calculate the centrality measures.

```
import networkx as nx
import sys


def parse_pajek(path):
    G = nx.Graph()
    labels = {}
    with open(path) as f:
        lines = [l.strip() for l in f if l.strip()]
    i = 0
    # skip to *vertices
    while i < len(lines) and not lines[i].lower().startswith("*vertices"):
        i += 1
    n = int(lines[i].split()[1])
    i += 1
    for _ in range(n):
        parts = lines[i].split()
        vid = int(parts[0])
        name = parts[1].strip('"')
        G.add_node(vid, label=name)
        labels[name] = vid
        i += 1
    # skip to *edges
    while i < len(lines) and not lines[i].lower().startswith("*edges"):
        i += 1
    i += 1
    while i < len(lines) and not lines[i].startswith("*"):
        parts = lines[i].split()
        G.add_edge(int(parts[0]), int(parts[1]))
        i += 1
    return G, labels


G, labels = parse_pajek("networks/dolphins.net")
target = "SN100"
tid = labels[target]

```

```python
print(f"Network: {G.number_of_nodes()} nodes, {G.number_of_edges()} edges\n")

# Degree centrality
dc = nx.degree_centrality(G)
dc_sorted = sorted(dc.items(), key=lambda x: -x[1])
rank_dc = next(i for i, (v, _) in enumerate(dc_sorted, 1) if v == tid)

# Betweenness centrality
bc = nx.betweenness_centrality(G)
bc_sorted = sorted(bc.items(), key=lambda x: -x[1])
rank_bc = next(i for i, (v, _) in enumerate(bc_sorted, 1) if v == tid)

# Closeness centrality
cc = nx.closeness_centrality(G)
cc_sorted = sorted(cc.items(), key=lambda x: -x[1])
rank_cc = next(i for i, (v, _) in enumerate(cc_sorted, 1) if v == tid)

# Eigenvector centrality
ec = nx.eigenvector_centrality(G, max_iter=1000)
ec_sorted = sorted(ec.items(), key=lambda x: -x[1])
rank_ec = next(i for i, (v, _) in enumerate(ec_sorted, 1) if v == tid)

print(f"=== {target} (node {tid}) ===")
print(
    f"  Degree:       {dc[tid]:.4f}  (deg={G.degree(tid)})  rank {rank_dc}/{G.number_of_nodes()}"
)
print(f"  Betweenness:  {bc[tid]:.4f}  rank {rank_bc}/{G.number_of_nodes()}")
print(f"  Closeness:    {cc[tid]:.4f}  rank {rank_cc}/{G.number_of_nodes()}")
print(f"  Eigenvector:  {ec[tid]:.4f}  rank {rank_ec}/{G.number_of_nodes()}")

print(f"\nNeighbors of {target}: {[G.nodes[v]['label'] for v in G.neighbors(tid)]}")

print("\n=== Top 10 by each centrality ===")
for name, ranked in [
    ("Degree", dc_sorted),
    ("Betweenness", bc_sorted),
    ("Closeness", cc_sorted),
    ("Eigenvector", ec_sorted),
]:
```

```
    print(f"\n{name}:")
    for v, val in ranked[:10]:
        marker = "␣<--" if v == tid else ""
        print(f"␣␣{G.nodes[v]['label']:15s}␣␣{val:.4f}{marker}")
```

2 HIV and network sampling

We approximate contact tracing via a simple random walk on the social network, stopping once 15% of nodes have been visited. The induced subgraph on the visited nodes forms our sampled network. Table 2 summarizes the structural properties of both networks.

Table 2: Structural properties of the original and sampled networks.

Property	Original	Sampled (15% RW)
Nodes	10680	1602
Edges	24316	8247
Average degree $\langle k \rangle$	4.55	10.30
Clustering C	0.2659	0.4580
C_{rand}	0.0004	0.0064
C/C_{rand}	623.69	71.21
Avg. shortest path L (GCC)	7.49	5.16
L_{rand}	6.12	3.16
L/L_{rand}	1.22	1.63
Power-law $\hat{\gamma}$	1.91	1.59
Power-law R^2	0.93	0.82

Original network. The clustering coefficient is $\sim$624$\times$ higher than expected for a random graph, while the average path length is only $\sim$1.2$\times$ the random baseline. This is characteristic of a small-world network. The degree distribution fits a power law with $\hat{\gamma} \approx 1.91$ and $R^2 = 0.93$, suggesting the network is also scale-free.

Sampled network. The sampled network remains small-world ($C \gg C_{\text{rand}}$, $L \approx L_{\text{rand}}$). However, the power-law fit degrades ($R^2 = 0.82$, $\hat{\gamma} \approx 1.59$). The random walk introduces a degree bias: high-degree nodes are visited disproportionately often (the stationary distribution of a random walk is proportional to node degree). This inflates the average degree from 4.55 to 10.30 and compresses the degree distribution, weakening the scale-free signature. The elevated clustering ($C = 0.46$ vs. 0.27) is also a consequence of this bias, the walk tends to stay within densely connected clusters.

The following is the Python implementation used.

```
import networkx as nx
from networkx import Graph
import random
import numpy as np
from collections import Counter
```

```python

random.seed(42)


def parse_pajek(path):
    G = nx.Graph()
    labels = {}
    with open(path) as f:
        lines = [l.strip() for l in f if l.strip()]
    i = 0
    while i < len(lines) and not lines[i].lower().startswith("*vertices"):
        i += 1
    n = int(lines[i].split()[1])
    i += 1
    for _ in range(n):
        parts = lines[i].split()
        vid = int(parts[0])
        name = parts[1].strip('"')
        G.add_node(vid, label=name)
        labels[name] = vid
        i += 1
    while i < len(lines) and not lines[i].lower().startswith("*edges"):
        i += 1
    i += 1
    while i < len(lines) and not lines[i].startswith("*"):
        parts = lines[i].split()
        G.add_edge(int(parts[0]), int(parts[1]))
        i += 1
    return G, labels


def random_walk(G: Graph, threshold: float):
    start_node = random.choice(list(G.nodes))
    rw = [start_node]
    visited = {start_node}

    while len(visited) < int(threshold * G.number_of_nodes()):
        next_node = random.choice(list(G.neighbors(rw[-1])))
        rw.append(next_node)
        visited.add(next_node)

    return rw


def analyze_network(G, name):
    print(f"\n=== {name} ===")
    print(f"Nodes: {G.number_of_nodes()}, Edges: {G.
```

```
        number_of_edges()}")

    avg_deg = 2 * G.number_of_edges() / G.number_of_nodes()
    print(f"Average degree: {avg_deg:.2f}")

    C = nx.average_clustering(G)
    print(f"Clustering coefficient: {C:.4f}")

    # Compare to random graph
    n = G.number_of_nodes()
    m = G.number_of_edges()
    p = 2 * m / (n * (n - 1))
    C_rand = p
    print(f"Expected C for random graph: {C_rand:.4f}")
    print(f"C / C_rand: {C / C_rand:.2f}")

    # Average shortest path length on largest connected component
    gcc = max(nx.connected_components(G), key=len)
    G_gcc = G.subgraph(gcc).copy()
    L = nx.average_shortest_path_length(G_gcc)
    L_rand = np.log(len(gcc)) / np.log(avg_deg) if avg_deg > 1
        else float('inf')
    print(f"Avg shortest path (GCC, n={len(gcc)}): {L:.4f}")
    print(f"Expected L for random graph: {L_rand:.4f}")
    print(f"L / L_rand: {L / L_rand:.2f}")

    small_world = C > 10 * C_rand and L / L_rand < 2
    print(f"Small-world: C >> C_rand and L ~ L_rand => {
        small_world}")

    # Degree distribution for scale-free check
    degrees = [d for _, d in G.degree()]
    deg_count = Counter(degrees)
    ks = sorted(deg_count.keys())
    # Log-log linear regression on CCDF
    ccdf = []
    total = len(degrees)
    for k in ks:
        ccdf.append(sum(1 for d in degrees if d >= k) / total)
    log_k = np.log(np.array(ks, dtype=float))
    log_ccdf = np.log(np.array(ccdf))
    coeffs = np.polyfit(log_k, log_ccdf, 1)
    gamma_est = -coeffs[0]
    residuals = log_ccdf - np.polyval(coeffs, log_k)
    ss_res = np.sum(residuals**2)
    ss_tot = np.sum((log_ccdf - np.mean(log_ccdf))**2)
    r_squared = 1 - ss_res / ss_tot
    print(f"Power-law fit: gamma ~ {gamma_est:.2f}, R^2 = {
        r_squared:.4f}")
```

```
    print(f"Max degree: {max(degrees)}, Min degree: {min(degrees)}")

    return {
        "n": n, "m": m, "avg_deg": avg_deg,
        "C": C, "C_rand": C_rand,
        "L": L, "L_rand": L_rand,
        "gamma": gamma_est, "r2": r_squared,
    }


G, labels = parse_pajek("networks/social.net")
rw = random_walk(G, 0.15)
visited = set(rw)
G_sampled = G.subgraph(visited).copy()

orig = analyze_network(G, "Original social network")
samp = analyze_network(G_sampled, "Sampled network (15% RW)")
```

3 Ring graph modularity

We calculate Q as follows

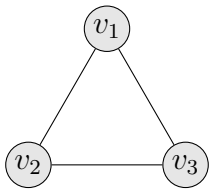
$$Q = \sum_{s=1}^{c} \left[\frac{l_s}{m} - \left(\frac{d_s}{2m} \right)^2 \right] \tag{1}$$

where for each cluster s:

- l_s = number of edges within cluster s.
- d_s = sum of degrees of all nodes in cluster s
- m = total number of edges in the graph.

Proposition 1. In a ring graph G with $n = |V(G)| \geq 3$, it holds that $n = m = |E(G)|$.

Proof. We follow by induction on n.
Base case ($n = 3$). Trivially follows from the illustration below.



Induction step. We argue the statement holds for $n + 1$. Let G be a ring graph with n vertices, then by the induction hypothesis it has exactly n edges. We add a vertex z to this graph and connect it to an arbitrary neighbouring vertex pair which w.l.o.g. we denote x, y. To adhere to the constraints of the ring graph, we must partition the edge xy into two parts, xz and zy. The new graph has exactly $n + 1$ vertices, and $n + 1$ edges. □

Proposition 2. In a ring graph G partitioned into $c \geq 2$ consecutive clusters, $l_s = n_c - 1$ for every cluster s.

Proof. Since $c \geq 2$, each cluster consists of n_c consecutive nodes on the ring and the induced subgraph is a path P_{n_c} on n_c vertices. A path on k vertices has $k - 1$ edges, so $l_s = |E(P_{n_c})| = n_c - 1$. □

Proposition 3. In a ring graph G, $d_s = 2n_c$.

Proof. In a ring graph all vertices have degree 2, due to the graph construction constraints. This implies that the sum of degrees of all nodes in a cluster s is equal to $2n_c$. □

Theorem 4. For a ring graph G partitioned into $c \geq 2$ consecutive clusters of size n_c, the modularity can be expressed as

$$Q = 1 - \frac{1}{n_c} - \frac{n_c}{n}.$$

Proof. Substituting Propositions 1–3 into Eq. 1, the summand is independent of s, so

$$Q = \sum_{s=1}^{c} \left[\frac{n_c - 1}{n} - \left(\frac{n_c}{n}\right)^2 \right] = c \left[\frac{n_c - 1}{n} - \frac{n_c^2}{n^2} \right].$$

Using $c = n/n_c$,

$$Q = \frac{n}{n_c} \left[\frac{n_c - 1}{n} - \frac{n_c^2}{n^2} \right] = \frac{n_c - 1}{n_c} - \frac{n_c}{n} = 1 - \frac{1}{n_c} - \frac{n_c}{n}.$$

□

We search for the cluster size n_c that maximizes the modularity of such a ring graph, i.e.

$$n_c^\star = \arg\max_{n_c \,|\, n} Q(n_c) = \arg\max_{n_c \,|\, n} \left[1 - \frac{1}{n_c} - \frac{n_c}{n} \right],$$

where $n_c \in \{2, \ldots, n\}$ and $n_c \mid n$.

Theorem 5. Let n be a perfect square with $n \geq 4$. The unique maximizer of Q over the set of divisors $\mathcal{D} = \{d \in \mathbb{Z} : d \mid n,\ 2 \leq d \leq n\}$ is

$$n_c^\star = \sqrt{n}, \qquad \text{with optimal value} \qquad Q^\star = 1 - \frac{2}{\sqrt{n}}.$$

Proof. Define the continuous relaxation $f : (0, \infty) \to \mathbb{R}$ by

$$f(x) = 1 - \frac{1}{x} - \frac{x}{n},$$

so that $Q(n_c) = f(n_c)$ for every $n_c \in \mathcal{D}$. The function f is twice differentiable on $(0, \infty)$ with

$$f'(x) = \frac{1}{x^2} - \frac{1}{n}, \qquad f''(x) = -\frac{2}{x^3}.$$

Since $f''(x) < 0$ for all $x > 0$, f is strictly concave on $(0, \infty)$.
Setting $f'(x) = 0$ gives $x^2 = n$, hence $x = \sqrt{n}$ (the negative root is discarded). By strict concavity this is the unique global maximizer of f on $(0, \infty)$, and

$$f(\sqrt{n}) = 1 - \frac{1}{\sqrt{n}} - \frac{\sqrt{n}}{n} = 1 - \frac{2}{\sqrt{n}}.$$

Since n is a perfect square, $\sqrt{n} \in \mathbb{Z}$. Moreover $\sqrt{n} \mid n$, since $n = \sqrt{n} \cdot \sqrt{n}$. For $n \geq 4$ we have $\sqrt{n} \geq 2$, so $\sqrt{n} \in \mathcal{D}$.
For every $n_c \in \mathcal{D}$, $Q(n_c) = f(n_c)$. Since $\sqrt{n}$ is the unique maximizer of f on $(0, \infty) \supseteq \mathcal{D}$, it follows that $\sqrt{n}$ is the unique maximizer of Q on $\mathcal{D}$. □

Corollary 6. The optimal partition itself is *not* unique: any cyclic rotation of the cut points yields a distinct partition achieving $Q^\star$. Concretely, there are exactly $n_c^\star = \sqrt{n}$ rotationally distinct optima.

Proof. Label the vertices $0, 1, \ldots, n-1$ around the ring. The cyclic group $\mathbb{Z}/n\mathbb{Z}$ acts on the vertex set by $i \mapsto i + r \pmod{n}$, and this action preserves the edge set, vertex degrees, and cluster sizes. Hence rotating any optimum partition produces another partition with the same value of Q. The stabilizer of an optimum partition consists of rotations by multiples of $n_c^\star$, a subgroup of order $c = n/n_c^\star = \sqrt{n}$. By the orbit–stabilizer theorem, the orbit has size $|\mathbb{Z}/n\mathbb{Z}|/\sqrt{n} = n/\sqrt{n} = \sqrt{n}$ distinct partitions. □

Remark 7. Theorem 5 gives $Q^\star = 1 - 2/\sqrt{n} \to 1$ as $n \to \infty$, despite the ring being a vertex-transitive graph with no genuine community structure. Combined with Corollary 6, this illustrates a known pathology of modularity optimization: it returns spurious, highly-modular partitions even on homogeneous graphs, and the optimum is degenerate under symmetries of the underlying graph. Applying modularity optimization to a ring graph is therefore not particularly meaningful.

4 Who's the winner?

We compare three community detection algorithms drawn from different paradigms:

- **Leiden** [TWVE19] – modularity optimization (`leidenalg`).
- **FLPA** – fast label propagation (NetworkX).
- **Infomap** – map equation / random walks (`python-igraph`).

For each experiment we run 25 graph realizations per parameter value and report the average.

(i) Girvan–Newman benchmark

Three groups of 24 nodes with expected node degree $\langle k \rangle = 20$. The mixing parameter μ controls the fraction of expected edges placed between groups; the within- and between-group edge probabilities are

$$p_{\text{in}} = \frac{(1-\mu)\langle k \rangle}{n_g - 1}, \qquad p_{\text{out}} = \frac{\mu \langle k \rangle}{n_g(c-1)},$$

which keeps the expected degree of each node equal to $\langle k \rangle$ regardless of μ.

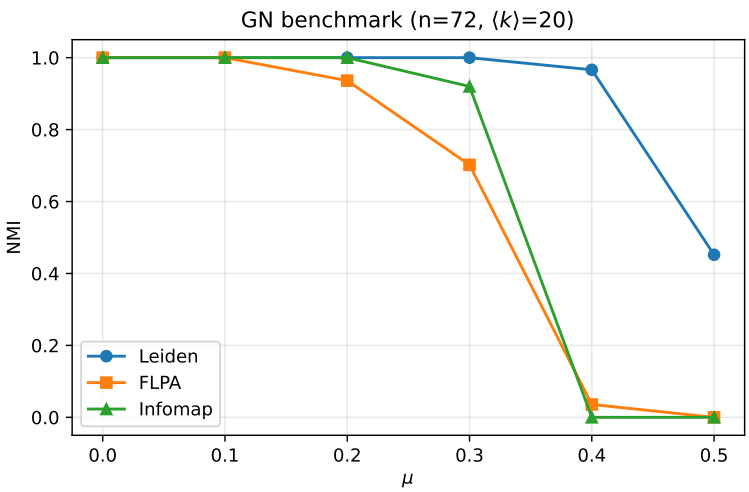


Figure 1: Average NMI over 25 GN benchmark realizations vs. mixing parameter μ.

Discussion. Leiden is the clear winner: it stays at NMI ≥ 0.97 up through $\mu = 0.4$ and only drops to 0.45 at the detectability threshold $\mu = 0.5$, where the

within- and between-group expected degrees are nearly equal. Infomap matches Leiden up to $\mu = 0.3$ but collapses sharply at $\mu = 0.4$, its random walks no longer get trapped inside the planted groups. FLPA degrades earliest because label propagation has no global optimization step; once between-group ties are dense enough, labels leak across groups during a single sweep.

The GN benchmark code is given below.

```
"""Girvan-Newman planted-partition benchmark graph generator.

Each node has expected degree ``k_avg``. A fraction ``1 - mu`` of its expected
degree is allocated to within-group neighbours and a fraction ``mu`` to
between-group neighbours. The within- and between-group edge probabilities are
chosen so that the expected total degree of every node equals ``k_avg``
regardless of ``mu``.
"""

from __future__ import annotations

import numpy as np
import networkx as nx


def gn_benchmark(
    n_groups: int = 3,
    group_size: int = 24,
    k_avg: float = 20.0,
    mu: float = 0.0,
    seed: int | None = None,
) -> tuple[nx.Graph, dict[int, int]]:
    """Generate one realisation of the GN planted-partition benchmark.

    Parameters
    ----------
    n_groups, group_size
        The graph has ``n_groups * group_size`` nodes split into equal groups.
    k_avg
        Expected node degree. Must satisfy
        ``k_avg <= group_size - 1 + group_size * (n_groups - 1)``.
    mu
        Mixing parameter in ``[0, 1]``. ``mu = 0`` -> all edges within groups,
        ``mu = 1`` -> all edges between groups.
```

```
␣␣␣␣seed
␣␣␣␣␣␣␣␣Seed␣for␣reproducibility.

␣␣␣␣Returns
␣␣␣␣-------
␣␣␣␣G
␣␣␣␣␣␣␣␣The␣generated␣graph␣on␣``n␣=␣n_groups␣*␣group_size``␣nodes.
␣␣␣␣membership
␣␣␣␣␣␣␣␣Mapping␣``node␣->␣group␣index``.
␣␣␣␣"""
    if not 0.0 <= mu <= 1.0:
        raise ValueError("mu␣must␣lie␣in␣[0,␣1]")

    rng = np.random.default_rng(seed)
    n = n_groups * group_size

    # Within-group / between-group neighbour counts per node.
    n_in = group_size - 1
    n_out = group_size * (n_groups - 1)

    # Edge probabilities chosen so that E[degree] = k_avg for every node.
    p_in = (1.0 - mu) * k_avg / n_in
    p_out = mu * k_avg / n_out
    if not (0.0 <= p_in <= 1.0 and 0.0 <= p_out <= 1.0):
        raise ValueError(f"Edge␣probabilities␣out␣of␣[0,1]:␣p_in={p_in},␣p_out={p_out}")

    membership = {v: v // group_size for v in range(n)}

    G = nx.Graph()
    G.add_nodes_from(range(n))
    for u in range(n):
        for v in range(u + 1, n):
            same = membership[u] == membership[v]
            p = p_in if same else p_out
            if rng.random() < p:
                G.add_edge(u, v)
    return G, membership


if __name__ == "__main__":
    G, mem = gn_benchmark(mu=0.2, seed=0)
    print(f"n={G.number_of_nodes()},␣m={G.number_of_edges()},␣"
          f"<k>={2␣*␣G.number_of_edges()␣/␣G.number_of_nodes():.2f}")
```

(ii) LFR benchmark

LFR graphs with $n = 2500$ nodes, power-law degree exponent $\tau_1 = 2.5$, community-size exponent $\tau_2 = 1.5$, average degree 20, max degree 50, and community sizes in $[20, 100]$.

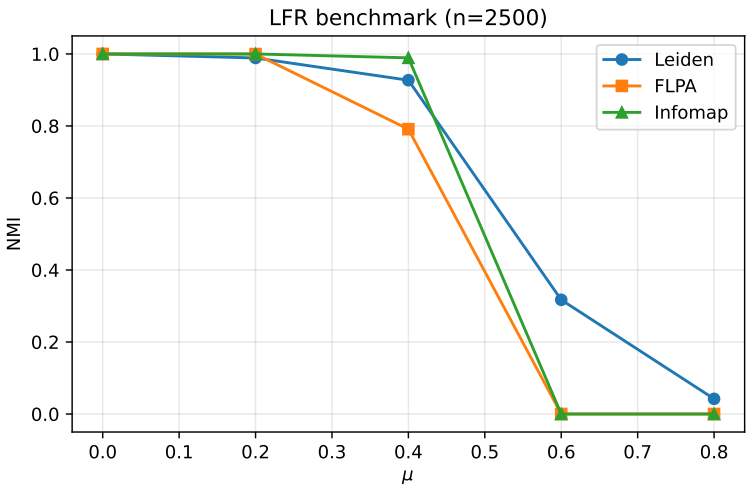


Figure 2: Average NMI over 25 LFR realizations vs. mixing parameter μ.

Discussion. The picture is genuinely different from (i). At $\mu \in \{0.2, 0.4\}$ **Infomap** is the best (NMI ≈ 1.0 and 0.99), beating Leiden. This is because LFR has heterogeneous community sizes: Leiden's modularity score has a known resolution limit and tends to merge small communities, while Infomap's compression objective is essentially size-agnostic. However, both Infomap and FLPA collapse to NMI ≈ 0 at $\mu = 0.6$, whereas Leiden retains a nontrivial signal (NMI ≈ 0.32). At $\mu = 0.8$ all three fail. So Infomap wins in the easy/intermediate regime but Leiden is the most robust into the high-mixing regime.

(iii) Erdős–Rényi robustness

Random graphs with $n = 1000$ and average degree $\langle k \rangle \in \{8, 16, 24, 32, 40\}$. The ground truth is the connected-component partition; we report normalized variation of information $\mathrm{NVI}(X, Y) = \mathrm{VI}(X, Y)/\log n \in [0, 1]$ between the algorithm output and the components.

Discussion. FLPA is fully robust: NVI ≈ 0 for all densities, when there is no community signal, label propagation just collapses everything to one label per component. **Infomap** is robust for $\langle k \rangle \geq 16$ (NVI $= 0$), and only struggles in the very sparse regime $\langle k \rangle = 8$ where the graph has many small components and

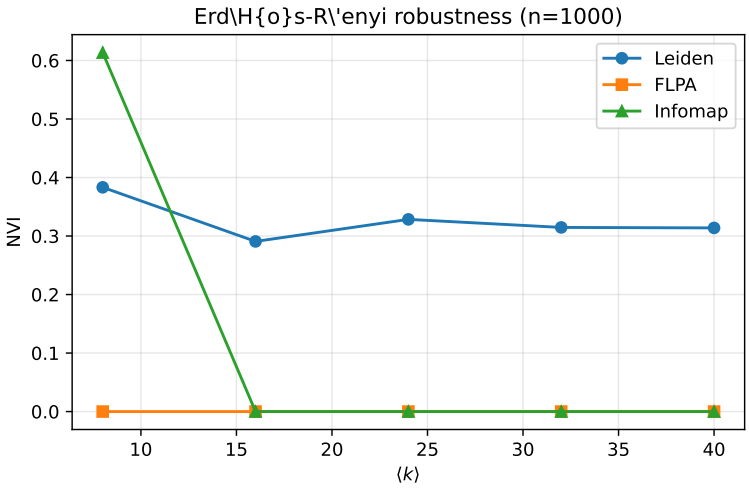


Figure 3: Average NVI over 25 ER realizations vs. average degree (lower is more robust).

the random walk gets trapped in micro-clusters. **Leiden is the least robust**: it consistently reports NVI ≈ 0.3, hallucinating communities at every density. This is the same modularity pathology we saw analytically on the ring graph in Section 5, modularity has a positive maximum even on graphs with no community structure, so Leiden returns a non-trivial partition by construction.

Summary. Leiden wins on GN, Infomap wins on LFR (in the detectable regime), FLPA and Infomap win on ER robustness. There is no winner, the right algorithm depends on whether community sizes are homogeneous (favours Leiden), heterogeneous (favours Infomap), or whether one is concerned about false positives on structureless graphs (avoid Leiden).

The full implementation of the experiments is given below.

```
"""Run the three community-detection experiments for HW2 Section 4.

Algorithms compared:
    * Leiden (modularity, via :mod:`leidenalg`)
    * FLPA (fast label propagation, via NetworkX)
    * Infomap (map equation, via :mod:`igraph`)

Experiments:
    (i)   Girvan-Newman planted partition  ->  NMI vs mu
    (ii)  LFR benchmark (n = 2500)         ->  NMI vs mu
    (iii) Erdos-Renyi (no community)       ->  NVI vs <k>
"""

```

```python
from __future__ import annotations

import math
from collections import Counter
from pathlib import Path

import matplotlib.pyplot as plt
import networkx as nx
import numpy as np
from networkx.algorithms.community.label_propagation import (
    fast_label_propagation_communities,
)
from networkx.generators.community import LFR_benchmark_graph
from sklearn.metrics import normalized_mutual_info_score

import igraph as ig
import leidenalg

from gn_benchmark import gn_benchmark


# Algorithms.  Each takes a NetworkX graph and returns an integer label per
# node, indexed by the node id (which is always 0..n-1 in our experiments).


def _nx_to_igraph(G: nx.Graph) -> ig.Graph:
    n = G.number_of_nodes()
    edges = [(u, v) for u, v in G.edges() if u != v]
    g = ig.Graph(n=n, edges=edges, directed=False)
    return g


def _membership_to_labels(membership, n: int) -> np.ndarray:
    """Convert an iterable of clusters (each a collection of node ids) or a
    flat per-vertex membership list to a numpy array of shape (n,)."""
    labels = np.full(n, -1, dtype=int)
    if hasattr(membership, "__iter__") and not isinstance(membership, (list, tuple)):
        membership = list(membership)
    # flat case: list/array with one entry per vertex of integer type
    if (
        isinstance(membership, (list, tuple, np.ndarray))
        and len(membership) == n
        and np.isscalar(membership[0])
```

```
    ):
        return np.asarray(membership, dtype=int)
    # cluster-of-nodes case
    for c_idx, cluster in enumerate(membership):
        for v in cluster:
            labels[int(v)] = c_idx
    if (labels == -1).any():
        # singleton fallback
        for i in np.where(labels == -1)[0]:
            labels[i] = labels.max() + 1
    return labels


def run_leiden(G: nx.Graph, seed: int | None = None) -> np.ndarray:
    g = _nx_to_igraph(G)
    part = leidenalg.find_partition(g, leidenalg.ModularityVertexPartition, seed=seed)
    return np.asarray(part.membership, dtype=int)


def run_flpa(G: nx.Graph, seed: int | None = None) -> np.ndarray:
    communities = fast_label_propagation_communities(G, seed=seed)
    return _membership_to_labels(list(communities), G.number_of_nodes())


def run_infomap(G: nx.Graph, seed: int | None = None) -> np.ndarray:
    import random as _random

    g = _nx_to_igraph(G)
    if seed is not None:
        ig.set_random_number_generator(_random.Random(seed))
    part = g.community_infomap()
    return np.asarray(part.membership, dtype=int)


ALGORITHMS = {
    "Leiden": run_leiden,
    "FLPA": run_flpa,
    "Infomap": run_infomap,
}


# Metrics


def nmi(labels_true: np.ndarray, labels_pred: np.ndarray) -> float
```

```
    :
    """Normalised mutual information (arithmetic mean normalisation)."""
    return float(
        normalized_mutual_info_score(
            labels_true, labels_pred, average_method="arithmetic"
        )
    )


def nvi(labels_a: np.ndarray, labels_b: np.ndarray) -> float:
    """Normalised variation of information, NVI = VI / log(n) in [0, 1]."""
    n = len(labels_a)
    if n != len(labels_b):
        raise ValueError("labels must have the same length")

    joint = Counter(zip(labels_a.tolist(), labels_b.tolist()))
    px = Counter(labels_a.tolist())
    py = Counter(labels_b.tolist())

    def H(counts):
        return -sum((c / n) * math.log(c / n) for c in counts.values() if c > 0)

    Hx, Hy = H(px), H(py)
    Ixy = 0.0
    for (x, y), c in joint.items():
        pxy = c / n
        Ixy += pxy * math.log(pxy / ((px[x] / n) * (py[y] / n)))
    vi = Hx + Hy - 2.0 * Ixy
    return float(vi / math.log(n))


# Helpers


def cc_labels(G: nx.Graph) -> np.ndarray:
    """Return a label per node giving its connected-component index."""
    labels = np.zeros(G.number_of_nodes(), dtype=int)
    for c_idx, comp in enumerate(nx.connected_components(G)):
        for v in comp:
            labels[v] = c_idx
    return labels


def make_lfr(n: int, mu: float, seed: int) -> nx.Graph:
    """Generate an LFR benchmark graph; retries a few seeds on
```

```
        failure."""
    last_err = None
    for attempt in range(20):
        try:
            G = LFR_benchmark_graph(
                n=n,
                tau1=2.5,
                tau2=1.5,
                mu=max(mu, 1e-4),  # mu = 0 fails inside networkx LFR
                average_degree=20,
                max_degree=50,
                min_community=20,
                max_community=100,
                seed=seed + attempt * 1000,
            )
            return G
        except (
            nx.ExceededMaxIterations,
            RuntimeError,
            Exception,
        ) as err:  # noqa: BLE001
            last_err = err
    raise RuntimeError(f"LFR generation failed for mu={mu}: {last_err}")


def lfr_planted_labels(G: nx.Graph) -> np.ndarray:
    """Extract the planted partition stored on LFR graph nodes."""
    n = G.number_of_nodes()
    labels = np.full(n, -1, dtype=int)
    comm_id: dict[frozenset, int] = {}
    for v in G.nodes():
        c = frozenset(G.nodes[v]["community"])
        if c not in comm_id:
            comm_id[c] = len(comm_id)
        labels[v] = comm_id[c]
    return labels


# Experiments

N_REPS = 25
OUT_DIR = Path(__file__).parent / "networks"
OUT_DIR.mkdir(exist_ok=True)


def experiment_gn() -> dict[str, list[float]]:
    mus = [0.0, 0.1, 0.2, 0.3, 0.4, 0.5]
```

```
    results: dict[str, list[float]] = {a: [] for a in ALGORITHMS}
    for mu in mus:
        per_algo = {a: [] for a in ALGORITHMS}
        for r in range(N_REPS):
            seed = 1000 * int(round(mu * 10)) + r
            G, membership = gn_benchmark(mu=mu, seed=seed)
            true_labels = np.asarray(
                [membership[v] for v in range(G.number_of_nodes())]
            )
            for name, algo in ALGORITHMS.items():
                pred = algo(G, seed=seed)
                per_algo[name].append(nmi(true_labels, pred))
        for name in ALGORITHMS:
            results[name].append(float(np.mean(per_algo[name])))
            print(f"[GN]  mu={mu:.1f}  {name:7s}  NMI={results[name][-1]:.3f}")
    return mus, results


def experiment_lfr() -> dict[str, list[float]]:
    mus = [0.0, 0.2, 0.4, 0.6, 0.8]
    results: dict[str, list[float]] = {a: [] for a in ALGORITHMS}
    for mu in mus:
        per_algo = {a: [] for a in ALGORITHMS}
        for r in range(N_REPS):
            seed = 2000 * int(round(mu * 10)) + r
            G = make_lfr(n=2500, mu=mu, seed=seed)
            true_labels = lfr_planted_labels(G)
            G_int = nx.convert_node_labels_to_integers(G, ordering="sorted")
            for name, algo in ALGORITHMS.items():
                pred = algo(G_int, seed=seed)
                per_algo[name].append(nmi(true_labels, pred))
        for name in ALGORITHMS:
            results[name].append(float(np.mean(per_algo[name])))
            print(f"[LFR] mu={mu:.1f}  {name:7s}  NMI={results[name][-1]:.3f}")
    return mus, results


def experiment_er() -> dict[str, list[float]]:
    avg_ks = [8, 16, 24, 32, 40]
    n = 1000
    results: dict[str, list[float]] = {a: [] for a in ALGORITHMS}
    for k_avg in avg_ks:
        p = k_avg / (n - 1)
        per_algo = {a: [] for a in ALGORITHMS}
        for r in range(N_REPS):
```

```
            seed = 100 * k_avg + r
            G = nx.fast_gnp_random_graph(n=n, p=p, seed=seed)
            cc = cc_labels(G)
            for name, algo in ALGORITHMS.items():
                pred = algo(G, seed=seed)
                per_algo[name].append(nvi(cc, pred))
        for name in ALGORITHMS:
            results[name].append(float(np.mean(per_algo[name])))
            print(f"[ER]  <k>={k_avg:>2d}  {name:7s}  NVI={results[name][-1]:.3f}")
    return avg_ks, results


# Plotting

PLOT_DIR = Path(__file__).parent / "plots"
PLOT_DIR.mkdir(exist_ok=True)


def plot_results(xs, results, xlabel: str, ylabel: str, title: str, fname: str) -> None:
    plt.figure(figsize=(6, 4))
    markers = {"Leiden": "o", "FLPA": "s", "Infomap": "^"}
    for name, ys in results.items():
        plt.plot(xs, ys, marker=markers.get(name, "o"), label=name, linewidth=1.6)
    plt.xlabel(xlabel)
    plt.ylabel(ylabel)
    plt.title(title)
    plt.grid(True, alpha=0.3)
    plt.legend()
    plt.tight_layout()
    plt.savefig(PLOT_DIR / fname, dpi=150)
    plt.close()


def main() -> None:
    print("=== (i) Girvan-Newman benchmark ===")
    gn_mus, gn_res = experiment_gn()
    plot_results(
        gn_mus,
        gn_res,
        xlabel=r"$\mu$",
        ylabel="NMI",
        title="GN benchmark (n=72, $\\langle k \\rangle$=20)",
        fname="gn_nmi.pdf",
    )

    print("\n=== (ii) LFR benchmark ===")
```

```python
    lfr_mus, lfr_res = experiment_lfr()
    plot_results(
        lfr_mus,
        lfr_res,
        xlabel=r"$\mu$",
        ylabel="NMI",
        title="LFR benchmark (n=2500)",
        fname="lfr_nmi.pdf",
    )

    print("\n=== (iii) Erdos-Renyi robustness ===")
    er_ks, er_res = experiment_er()
    plot_results(
        er_ks,
        er_res,
        xlabel=r"$\langle k \rangle$",
        ylabel="NVI",
        title=r"Erd\H{o}s-R\'enyi robustness (n=1000)",
        fname="er_nvi.pdf",
    )

    print("\nWrote plots to", PLOT_DIR)
    # Also dump results to a small text file for reference.
    with open(PLOT_DIR / "results.txt", "w") as fh:
        fh.write("# GN benchmark NMI\n")
        fh.write("mu\t" + "\t".join(gn_res) + "\n")
        for i, mu in enumerate(gn_mus):
            fh.write(
                f"{mu}\t" + "\t".join(f"{gn_res[a][i]:.4f}" for a in gn_res) + "\n"
            )
        fh.write("\n# LFR benchmark NMI\n")
        fh.write("mu\t" + "\t".join(lfr_res) + "\n")
        for i, mu in enumerate(lfr_mus):
            fh.write(
                f"{mu}\t" + "\t".join(f"{lfr_res[a][i]:.4f}" for a in lfr_res) + "\n"
            )
        fh.write("\n# ER NVI\n")
        fh.write("k_avg\t" + "\t".join(er_res) + "\n")
        for i, k in enumerate(er_ks):
            fh.write(f"{k}\t" + "\t".join(f"{er_res[a][i]:.4f}" for a in er_res) + "\n")


if __name__ == "__main__":
    main()
```

5 Get at least 70% right

We are given an APS citation network of $n = 90{,}162$ papers and 429,650 directed citation arcs, published in 10 journals between 2008 and 2013. The training set consists of all 74,467 papers from 2008-2012 (journal known); the test set is the 15,695 papers from 2013.

Strategy: citation-profile multinomial logistic regression

For each paper v we extract a 30-dimensional feature vector that summarises the journal mix of the labelled papers v cites (i.e. its out-neighbours that fall in 2008-2012). Concretely, let $c_{v,j}$ be the count of v's labelled out-neighbours that belong to journal $j \in \{1, \ldots, 10\}$. The feature vector is the concatenation

$$\phi(v) = \left[\, c_{v,1}, \ldots, c_{v,10} \;\middle|\; \log(1 + c_{v,1}), \ldots, \log(1 + c_{v,10}) \;\middle|\; p_{v,1}, \ldots, p_{v,10} \,\right] \in \mathbb{R}^{30},$$

where $p_{v,j} = c_{v,j} / \sum_{j'} c_{v,j'}$ is the citation proportion. A multinomial logistic regression is then fit on $\{(\phi(v), y_v) : v \in \text{2008-2012}\}$ using the SAGA solver, and its predictions on $\{\phi(v) : v \in 2013\}$ are taken as the predicted journals.

Motivation

Each APS journal has a characteristic citation pattern: a Phys. Rev. B paper cites mostly Phys. Rev. B, while a Phys. Rev. Letters paper draws from a broader pool. The plain majority-vote baseline only uses the *mode* of this distribution and discards the rest. Logistic regression instead lets the entire citation profile contribute, weighting each journal by a learned coefficient. The three feature blocks complement each other:

- raw counts encode citation *volume* per journal;
- $\log(1 + c)$ compresses heavy tails so a single highly-cited review paper does not dominate;
- proportions remove the absolute scale and isolate the *shape* of the profile.

Out-edges (papers cited by v) are used because they are guaranteed labelled (older papers); in-edges from 2013 are mostly unlabelled and we found they only add noise.

Table 3: Test-set classification accuracy on the 15,695 papers from 2013.

Method	Accuracy
Baseline (most frequent journal among labelled neighbours)	0.6722
Citation-profile LR (mean of 10 runs, std $< 10^{-4}$)	**0.7216**
Improvement over baseline	+4.94 pp

Results

Per-run accuracies were $\{0.7215, 0.7215, 0.7216, 0.7216, 0.7216, 0.7216, 0.7216, 0.7216, 0.7216, 0.7216\}$ across 10 SAGA seeds, so the strategy is essentially deterministic up to floating-point noise. The baseline reproduces the spec's $\approx 67\%$ figure exactly, and the strategy exceeds the $\geq 70\%$ target by roughly two percentage points.

```
from __future__ import annotations

import re
import warnings
from collections import Counter
from pathlib import Path

import numpy as np
from sklearn.exceptions import ConvergenceWarning
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

warnings.filterwarnings("ignore", category=ConvergenceWarning)
warnings.filterwarnings("ignore", category=RuntimeWarning)

NET_PATH = Path(__file__).parent / "networks" / "aps_2008_2013.net"
N_JOURNALS = 10
TRAIN_YEARS = (2008, 2009, 2010, 2011, 2012)
TEST_YEAR = 2013


# I/O


def load_pajek(path: Path = NET_PATH):
    """Parse the Pajek file into adjacency lists and per-node attributes."""
    with open(path) as f:
        n = int(f.readline().split()[1])
        journal = np.zeros(n + 1, dtype=np.int8)
```

```
        year = np.zeros(n + 1, dtype=np.int16)
        year_pat = re.compile(r"-(\d{4})\b")
        for _ in range(n):
            parts = f.readline().split()
            vid = int(parts[0])
            label = parts[1].strip('"')
            journal[vid] = int(parts[2])
            m = year_pat.search(label)
            year[vid] = int(m.group(1)) if m else -1
        m_arcs = int(f.readline().split()[1])
        out_adj: list[list[int]] = [[] for _ in range(n + 1)]
        in_adj: list[list[int]] = [[] for _ in range(n + 1)]
        for line in f:
            parts = line.split()
            if not parts:
                continue
            u, v = int(parts[0]), int(parts[1])
            out_adj[u].append(v)
            in_adj[v].append(u)
    und_adj: list[list[int]] = [
        list({u for u in out_adj[v] + in_adj[v]}) for v in range(n
            + 1)
    ]
    return {
        "n": n,
        "journal": journal,
        "year": year,
        "out_adj": out_adj,
        "in_adj": in_adj,
        "und_adj": und_adj,
        "n_arcs": m_arcs,
    }


# Feature extraction


def citation_profile_features(data) -> np.ndarray:
    """Return an (n+1, 30) matrix of [counts | log1p(counts) |
        proportions]
    over the 10 journals, computed from each paper's labelled out-
    neighbours."""
    n = data["n"]
    journal = data["journal"]
    year = data["year"]
    out_adj = data["out_adj"]

    counts = np.zeros((n + 1, N_JOURNALS), dtype=np.float32)
    for v in range(1, n + 1):
```

```
        for u in out_adj[v]:
            if 2008 <= year[u] <= 2012 and 1 <= journal[u] <= N_JOURNALS:
                counts[v, journal[u] - 1] += 1.0
    log_counts = np.log1p(counts)
    row_sums = counts.sum(axis=1, keepdims=True)
    row_sums[row_sums == 0] = 1.0
    proportions = counts / row_sums
    return np.concatenate([counts, log_counts, proportions], axis=1)


# Baseline


def baseline_majority(data) -> np.ndarray:
    """Most frequent journal among labelled (2008-2012) neighbours.

    Tie-breaking: deterministic (smallest journal id wins) so the baseline is
    fully reproducible. Falls back to the global majority class for nodes
    that have no labelled neighbours."""
    journal = data["journal"]
    year = data["year"]
    und = data["und_adj"]
    n = data["n"]
    pred = np.zeros(n + 1, dtype=np.int8)
    global_majority = int(
        Counter(
            int(journal[v]) for v in range(1, n + 1) if 2008 <= year[v] <= 2012
        ).most_common(1)[0][0]
    )
    for v in range(1, n + 1):
        if year[v] != TEST_YEAR:
            continue
        cnt: Counter = Counter()
        for u in und[v]:
            if 2008 <= year[u] <= 2012:
                cnt[int(journal[u])] += 1
        if not cnt:
            pred[v] = global_majority
        else:
            top = max(cnt.values())
            pred[v] = min(j for j, c in cnt.items() if c == top)
    return pred

```

```python

# Strategy


def predict_lr(data, X: np.ndarray, random_state: int) -> np.ndarray:
    """Fit a multinomial LR on 2008-2012 nodes and predict 2013 nodes."""
    journal = data["journal"]
    year = data["year"]
    n = data["n"]

    train_idx = np.where((year >= 2008) & (year <= 2012))[0]
    test_idx = np.where(year == TEST_YEAR)[0]

    Xtr = X[train_idx]
    ytr = journal[train_idx]
    has_features = Xtr[:, :N_JOURNALS].sum(axis=1) > 0

    scaler = StandardScaler(with_mean=False).fit(Xtr[has_features])
    clf = LogisticRegression(
        solver="saga",
        max_iter=400,
        C=1.0,
        random_state=random_state,
        n_jobs=-1,
    ).fit(scaler.transform(Xtr[has_features]), ytr[has_features])

    pred = np.zeros(n + 1, dtype=np.int8)
    pred[test_idx] = clf.predict(scaler.transform(X[test_idx]))
    return pred


# Evaluation


def accuracy(pred: np.ndarray, data) -> float:
    journal = data["journal"]
    year = data["year"]
    test_mask = year == TEST_YEAR
    return float(np.mean(pred[test_mask] == journal[test_mask]))


def run_experiment(n_runs: int = 10) -> None:
    print("Loading network ...")
    data = load_pajek()
    n_train = int(((data["year"] >= 2008) & (data["year"] <= 2012)).sum())
```

```
    n_test = int((data["year"] == TEST_YEAR).sum())
    print(f"␣␣n_nodes␣=␣{data['n']:,},␣arcs␣=␣{data['n_arcs']:,}")
    print(f"␣␣train␣(2008-2012)␣=␣{n_train:,},␣test␣(2013)␣=␣{n_test:,}")

    print("\nBuilding␣citation-profile␣features␣...")
    X = citation_profile_features(data)
    print(f"␣␣feature␣matrix:␣shape={X.shape},␣dtype={X.dtype}")

    base_acc = accuracy(baseline_majority(data), data)
    print(f"\nBaseline␣(most-frequent␣labelled␣neighbour):␣{base_acc:.4f}")

    accs = []
    for r in range(n_runs):
        pred = predict_lr(data, X, random_state=1000 + r)
        a = accuracy(pred, data)
        accs.append(a)
        print(f"␣␣run␣{r+1:2d}:␣accuracy␣=␣{a:.4f}")

    print()
    print(
        f"Strategy␣mean␣accuracy␣over␣{n_runs}␣runs␣=␣"
        f"{np.mean(accs):.4f}␣␣(std␣{np.std(accs):.4f})"
    )
    print(
        f"Improvement␣over␣baseline␣=␣"
        f"{(np.mean(accs)␣-␣base_acc)␣*␣100:+.2f}␣percentage␣points"
    )


if __name__ == "__main__":
    run_experiment(n_runs=10)
```

6 Peers, ties and the Internet

We compare three link prediction indices that each isolate a different structural property:

- **Preferential attachment** [LNK07]: $s_{ij} = k_i\, k_j$.
- **Adamic–Adar** [AA03]: $s_{ij} = \sum_{x \in \Gamma_i \cap \Gamma_j} \frac{1}{\log k_x}$.
- **Community (block)**: with $\{c\}$ from Leiden modularity optimization [TWVE19] on the training graph,

$$s_{ij} = \begin{cases} m_c / \binom{n_c}{2}, & c_i = c_j = c, \\ m_{c_i,c_j} / (n_{c_i} n_{c_j}), & c_i \neq c_j, \end{cases}$$

  where m_c is the number of edges inside community c, $m_{c,c'}$ is the number of train edges between communities c and c', and n_c is the size of c. This is the maximum-likelihood block density of an SBM fitted to the Leiden partition.

Framework

Each run hides a uniformly random 10% of the edges as the test set, builds the training graph from the remaining 90%, runs Leiden on the training graph, and approximates AUC by Mann-Whitney sampling: draw $N = 10{,}000$ random pairs $(e_{\text{pos}}, e_{\text{neg}})$ where e_{pos} is a test edge and e_{neg} is a pair that is not an edge of the original graph; count $\hat{\text{AUC}} = (\#\{s(e_{\text{pos}}) > s(e_{\text{neg}})\} + \frac{1}{2}\#\{\text{ties}\})/N$. The four networks are:

- ER: $n = 25{,}000$, $\langle k \rangle = 10$ (regenerated each run).
- Circles: small Facebook social-circles sample ($n = 4{,}039$, $m = 88{,}234$).
- Gnutella: P2P file-sharing ($n = 62{,}586$, $m = 147{,}892$, undirected).
- nec: Internet AS-level overlay ($n = 75{,}885$, $m = 357{,}317$, undirected).

(x) Brief answer

Each network is won by the method whose structural assumption it most embodies. Erdős–Rényi has no exploitable structure, so all three indices score ≈ 0.5. The Facebook social-circles graph is dominated by triadic closure, so Adamic–Adar wins. Gnutella is scale-free but unclustered, so preferential attachment is the only useful signal. The AS-level Internet is hierarchically organised, so the community block index wins, with PA a strong runner-up because of Tier-1 hubs.

(z) AUC results and per-network discussion

Table 4: Mean AUC ± standard deviation over 10 runs. Best method per network in **bold**.

Network	PA	Adamic–Adar	Community
ER	0.502 ± 0.006	0.500 ± 0.000	0.497 ± 0.004
Circles	0.833 ± 0.004	$\mathbf{0.994 \pm 0.001}$	0.973 ± 0.002
Gnutella	$\mathbf{0.717 \pm 0.006}$	0.515 ± 0.001	0.510 ± 0.008
nec	0.823 ± 0.003	0.695 ± 0.007	$\mathbf{0.923 \pm 0.002}$

ER ($\langle k \rangle = 10$). Winner: none. All three indices score essentially 0.5, i.e. pure chance. ER has none of the three structural cues that the indices exploit: degrees are concentrated around the mean (no preferential-attachment signal), the clustering coefficient is $\approx p \approx 4 \times 10^{-4}$ (no triangles to feed Adamic–Adar), and Leiden hallucinates ≈ 17 communities of roughly equal block density (so the community score is uninformative, as we proved analytically for the ring graph in Section 5). The result confirms that none of the methods produce false positives on a structureless graph.

Circles (Facebook). Winner: Adamic–Adar. AA wins decisively at 0.994, with the community method close behind at 0.973. This is the prototypical social-network setting: a missing friendship between two people who already have many friends in common is overwhelmingly likely to actually exist (triadic closure). Communities are also tight, so the block index is almost as good. PA is the weakest of the three because in a small ego-network the degree variance is moderate, there are no extreme hubs, so $k_i k_j$ does not separate true links from random pairs nearly as well.

Gnutella P2P. Winner: preferential attachment. PA dominates (0.717) while Adamic–Adar and Community both collapse to near-random (0.515, 0.510). Gnutella has a heavily skewed degree distribution (a few super-peers and a long tail of leaf nodes) but extremely low clustering, i.e. P2P queries between strangers do not produce triangles, so the Adamic–Adar intersection is almost always empty. Leiden, in turn, produces $\approx 3{,}000$ tiny communities on the training graph, none of which capture meaningful structure, so the block density is essentially noise. Only the degree signal survives, and it is exactly preferential-attachment.

nec (Internet AS-level). Winner: community. The community index wins at 0.923, with PA in second at 0.823 and Adamic–Adar a distant third at 0.695. The AS-level Internet has a strong block / tier structure (regional ISPs, transit providers, customer cones) which Leiden recovers as ≈ 800 medium-sized communities, and these blocks are precisely where most missing links lie. PA

also performs well because the AS graph is famously scale-free with extreme hubs (Tier-1 carriers). Adamic–Adar is moderate but limited because the Internet is sparse on triangles compared to a social graph, transit relationships are structured rather than triadic.

(y) Implementation

```
from __future__ import annotations

import math
import random
import time
from collections import defaultdict
from pathlib import Path
from typing import Callable

import numpy as np
import networkx as nx

import igraph as ig
import leidenalg

NET_DIR = Path(__file__).parent / "networks"
N_RUNS = 10
TEST_FRAC = 0.1
N_AUC_SAMPLES = 10_000


# Loading


def load_pajek_undirected(path: Path) -> tuple[int, list[tuple[int, int]]]:
    """Parse a Pajek file and return (n, undirected edge list, 0-indexed)."""
    with open(path) as f:
        header = f.readline().split()
        n = int(header[1])
        for _ in range(n):
            f.readline()
        f.readline()  # *edges <m> or *arcs <m>
        edges: set[tuple[int, int]] = set()
        for line in f:
            parts = line.split()
            if len(parts) < 2:
                continue
            u = int(parts[0]) - 1
```

```
            v = int(parts[1]) - 1
            if u == v:
                continue
            edges.add((min(u, v), max(u, v)))
    return n, sorted(edges)


def make_er(n: int, k_avg: float, seed: int) -> tuple[int, list[tuple[int, int]]]:
    p = k_avg / (n - 1)
    G = nx.fast_gnp_random_graph(n, p, seed=seed)
    return n, [(min(u, v), max(u, v)) for u, v in G.edges()]


# Helpers


def split_edges(edges, frac, rng):
    e = list(edges)
    rng.shuffle(e)
    k = int(round(len(e) * frac))
    return e[k:], e[:k]


def neighbours_and_degree(n, train_edges):
    nbrs: list[set[int]] = [set() for _ in range(n)]
    for u, v in train_edges:
        nbrs[u].add(v)
        nbrs[v].add(u)
    deg = np.fromiter((len(s) for s in nbrs), dtype=np.int32, count=n)
    return nbrs, deg


def run_leiden(n, train_edges, seed):
    g = ig.Graph(n=n, edges=train_edges, directed=False)
    g = g.simplify()
    part = leidenalg.find_partition(g, leidenalg.ModularityVertexPartition, seed=seed)
    return np.asarray(part.membership, dtype=np.int32)


def community_stats(n, train_edges, membership):
    """Return (M, sizes) where M[(a,b)] is # train edges between community a and b
    (a <= b), and sizes[c] = number of nodes in community c."""
    sizes: dict[int, int] = defaultdict(int)
    for v in range(n):
        sizes[int(membership[v])] += 1
```

```
    M: dict[tuple[int, int], int] = defaultdict(int)
    for u, v in train_edges:
        a, b = int(membership[u]), int(membership[v])
        M[(min(a, b), max(a, b))] += 1
    return M, dict(sizes)


# Scores


def score_pa(deg):
    def f(u, v):
        return float(deg[u]) * float(deg[v])

    return f


def score_aa(nbrs, deg):
    log_term = np.zeros(len(deg), dtype=np.float64)
    for x, k in enumerate(deg):
        if k > 1:
            log_term[x] = 1.0 / math.log(float(k))

    def f(u, v):
        if len(nbrs[u]) > len(nbrs[v]):
            u, v = v, u
        s = 0.0
        nv = nbrs[v]
        for x in nbrs[u]:
            if x in nv:
                s += log_term[x]
        return s

    return f


def score_community(membership, M, sizes):
    def f(u, v):
        a = int(membership[u])
        b = int(membership[v])
        if a == b:
            n_a = sizes[a]
            denom = n_a * (n_a - 1) / 2.0
            return M.get((a, a), 0) / denom if denom > 0 else 0.0
        denom = sizes[a] * sizes[b]
        return M.get((min(a, b), max(a, b)), 0) / denom if denom >
            0 else 0.0

    return f
```

```python


# AUC by sampling


def sample_auc(score_fn, test_edges, full_edge_set, n, n_samples, rng):
    """Mann-Whitney AUC: draw n_samples (e_pos, e_neg) pairs."""
    test_arr = list(test_edges)
    hits = 0
    ties = 0
    for _ in range(n_samples):
        u_pos, v_pos = test_arr[rng.randrange(len(test_arr))]
        while True:
            u = rng.randrange(n)
            v = rng.randrange(n)
            if u == v:
                continue
            key = (u, v) if u < v else (v, u)
            if key in full_edge_set:
                continue
            break
        s_pos = score_fn(u_pos, v_pos)
        s_neg = score_fn(u, v)
        if s_pos > s_neg:
            hits += 1
        elif s_pos == s_neg:
            ties += 1
    return (hits + 0.5 * ties) / n_samples


# Per-network experiment


def run_network(name: str, loader: Callable, n_runs: int = N_RUNS) -> dict:
    print(f"\n=== {name} ===")
    t0 = time.time()
    n, edges = loader(seed=42)
    print(f"  n = {n:,}, m = {len(edges):,}  ({time.time()-t0:.1f}s)")
    full_edge_set = set(edges)
    results = {"PA": [], "AA": [], "Comm": []}

    for r in range(n_runs):
        run_t0 = time.time()
        rng = random.Random(7919 * (r + 1))
        # If ER, regenerate the graph for this run.
        if name == "ER":
```

```
            _, edges_run = loader(seed=10_000 + r)
            full_edge_set_run = set(edges_run)
        else:
            edges_run = edges
            full_edge_set_run = full_edge_set

        train, test = split_edges(edges_run, TEST_FRAC, rng)
        nbrs, deg = neighbours_and_degree(n, train)
        membership = run_leiden(n, train, seed=r + 1)
        M, sizes = community_stats(n, train, membership)

        f_pa = score_pa(deg)
        f_aa = score_aa(nbrs, deg)
        f_co = score_community(membership, M, sizes)

        rng_pa = random.Random(rng.randrange(1 << 30))
        rng_aa = random.Random(rng.randrange(1 << 30))
        rng_co = random.Random(rng.randrange(1 << 30))

        auc_pa = sample_auc(f_pa, test, full_edge_set_run, n,
            N_AUC_SAMPLES, rng_pa)
        auc_aa = sample_auc(f_aa, test, full_edge_set_run, n,
            N_AUC_SAMPLES, rng_aa)
        auc_co = sample_auc(f_co, test, full_edge_set_run, n,
            N_AUC_SAMPLES, rng_co)

        results["PA"].append(auc_pa)
        results["AA"].append(auc_aa)
        results["Comm"].append(auc_co)
        n_comm = len(set(membership.tolist()))
        print(
            f"  run {r+1:2d}: PA={auc_pa:.3f}  AA={auc_aa:.3f}  
                Comm={auc_co:.3f}  "
            f"(n_comm={n_comm}, {time.time()-run_t0:.1f}s)"
        )

    return results


# Main


def loader_er(seed):
    return make_er(25_000, 10, seed)


def loader_pajek(filename):
    def f(seed=None):
        return load_pajek_undirected(NET_DIR / filename)
```

```python

    return f


def main():
    networks = [
        ("ER", loader_er),
        ("Circles", loader_pajek("circles.net")),
        ("Gnutella", loader_pajek("gnutella.net")),
        ("nec", loader_pajek("nec.net")),
    ]

    all_results: dict[str, dict[str, list[float]]] = {}
    for name, loader in networks:
        all_results[name] = run_network(name, loader)

    # Summary
    print("\n" + "=" * 72)
    print(f"{'Network':<10} {'PA mean':>12}  {'AA mean':>12}  {'Comm mean':>12}")
    print("-" * 72)
    for name, res in all_results.items():
        line = f"{name:<10}"
        for m in ("PA", "AA", "Comm"):
            mu = float(np.mean(res[m]))
            sd = float(np.std(res[m]))
            line += f"  {mu:.3f}+/-{sd:.3f}"
        print(line)
    print("=" * 72)

    out_path = Path(__file__).parent / "plots" / "link_prediction_results.txt"
    out_path.parent.mkdir(exist_ok=True)
    with open(out_path, "w") as fh:
        fh.write("network\tmethod\tmean\tstd\truns\n")
        for name, res in all_results.items():
            for m, vals in res.items():
                fh.write(
                    f"{name}\t{m}\t{np.mean(vals):.4f}\t{np.std(vals):.4f}\t"
                    f"{','.join(f'{v:.4f}' for v in vals)}\n"
                )
    print(f"\nWrote {out_path}")


if __name__ == "__main__":
    main()
```

Homework #2

The homework does not require a lot of writing but may require some thinking. It does not require a lot of processing power but may require efficient programming. It accounts for 12.5% of the course grade. Any questions or comments should be posted on Piazza.

Submission details

This homework is due on **May 1st** at 11:59pm. It must be submitted through (1) Gradescope (entry code **RVYW5D**) and (2) eUcilnica. (1) Submission to Gradescope should include answers to all questions, each on a separate page, which may also demand pseudocode, proofs, tables, plots, diagrams and/or other. (2) Submission to eUcilnica should include this cover sheet with signed honor code and all the programming code used to complete the exercises (preferably in `.py` format). The homework is considered submitted only when *both* (1) and (2) have been submitted. Failing to include the honor code in the submission will result in **10% deduction**. Failing to submit all the developed code will result in **10% deduction**.

Honor code

Students are strongly encouraged to discuss the homework with other classmates and form study groups. However, each student must then solve the homework by herself/himself without the help of others and should be able to redo the homework at a later time. In other words, students are encouraged to collaborate but should not copy from one another. Referring to any solutions obtained from classmates, course books, previous years, found online, AI tools or other is considered an honor code violation. Also, stating any part of the solutions in class or on Piazza is considered an honor code violation.

Any violation of the honor code will be reported to the **faculty disciplinary committee** and vice dean for education.

SID: _____

Full name: _____

Study group: _____

I understand and accept the honor code.

Signature: _____